Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Руководство по развёртыванию

Установка, обновление, резервное копирование · версия 0.5.0

28 октября 2026

© Alexander Yurasov

Содержание

1 О руководстве 3

2 Требования 4

3 Установка 5

4 Настройка 6

5 Почта 9

6 Единый вход 10

7 Интеграция с кадровой системой 11

8 Проверка 12

9 Обновление 13

10 Резервное копирование 14

10.1 Восстановление 14

10.2 Ежемесячная проверка восстановления 14

11 Обслуживание 15

12 Производительность 16

13 Мониторинг 17

14 Реплика и архив WAL 19

14.1 Переключение на реплику 19

14.2 Восстановление на момент времени 20

15 Устранение неполадок 21

1 О руководстве

Руководство предназначено для системного администратора, который устанавливает и обслуживает «Планировщик отпусков» на сервере организации: требования, установка, настройка, обновление, резервное копирование и восстановление, проверка работоспособности и устранение неполадок.

Архитектура стенда описана в «Описании архитектуры», конвейеры и порядок выпуска — в документе 7.

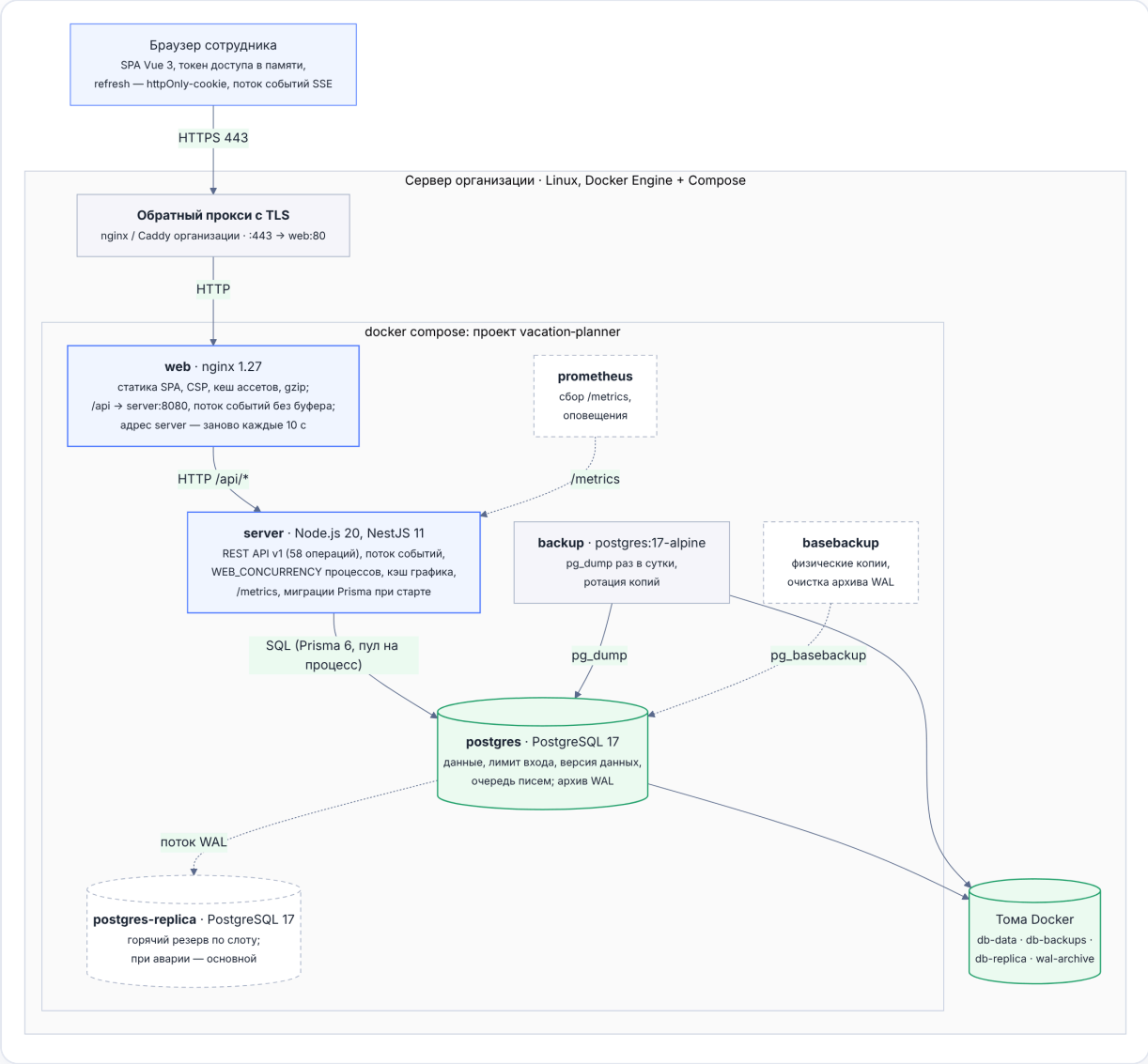


Рис. 1. Контейнеры стенда

2 Требования

РЕСУРС	ДО 1000 ПОЛЬЗОВАТЕЛЕЙ	ДО 5000 ПОЛЬЗОВАТЕЛЕЙ
Процессор	2 vCPU	4 vCPU под приложение; PostgreSQL — на отдельном узле или с репликой
Память	4 ГБ	8 ГБ на узел приложения
Диск	20 ГБ SSD (данные и копии за 14 дней)	50 ГБ SSD; архив WAL и физические копии — на отдельном диске или хранилище
ОС	Linux x86-64 с Docker Engine 24+ и Docker Compose v2	то же
Сеть	Обратный прокси организации с TLS на 443 → стенд на 80	то же; при втором экземпляре `server` — балансировщик

Расчёт для 5000 пользователей подтверждён нагрузочным прогоном: 500 одновременных пользователей, p95 чтения и записи в пределах требований, без ошибок (`docs/progress/load-v0.5.0.md`).

Клиенты — две последние версии Chrome, Edge, Firefox или Safari; на телефонах — мобильные браузеры.

3 Установка

1. Получите исходный код версии:

```
git clone <адрес репозитория> vacation-planner
cd vacation-planner
git checkout v0.5.0
```

2. Создайте файл `.env` рядом с `docker-compose.yml`:

```
# Секрет подписи токенов: случайная строка не короче 32 символов
JWT_SECRET=<результат openssl rand -base64 48>
# Резервные копии: интервал в секундах и срок хранения в днях
BACKUP_INTERVAL=86400
BACKUP_KEEP_DAYS=14
# Адрес планировщика для ссылок в письмах и единого входа
APP_URL=https://otpusk.company.ru
# Процессы API: auto — по числу ядер (по умолчанию)
WEB_CONCURRENCY=auto
```

Почта и единый вход необязательны — разделы «Почта» и «Единый вход» ниже.

3. Запустите стенд:

```
docker compose --profile full up -d --build
```

При первом старте контейнер `server` применяет миграции базы данных. Стенд отвечает на порту 80; прокси организации публикует его по HTTPS.

4. Создайте первого администратора. Для демонстрационного стенда загрузите демо-данные (131 сотрудник; администратор `yurasov.av`):

```
docker compose exec server npm run db:seed:prod
```

Для рабочего контура демо-данные не загружайте: создайте администратора SQL-скриптом из раздела «Обслуживание», затем остальных сотрудников — в интерфейсе.

5. Проверьте работоспособность (раздел «Проверка») и смените пароль администратора.

4 Настройка

ПЕРЕМЕННАЯ	ГДЕ	НАЗНАЧЕНИЕ	ПО УМОЛЧАНИЮ
`JWT_SECRET`	`server`	Секрет подписи access-токенов; не короче 32 символов, иначе сервер не стартует	—
`DATABASE_URL`	`server`	Строка подключения PostgreSQL; параметр `connection_limit` задаёт размер пула на процесс	из compose
`DB_HOST`	`server`, `backup`, `basebackup`	Узел БД в compose; при переключении на реплику — `postgres-replica`	`postgres`
`WEB_CONCURRENCY`	`server`	Число процессов API: число или `auto` — по числу ядер	`auto` в compose
`DB_POOL_SIZE`	`server`	Соединений с БД на процесс, если `connection_limit` не задан в `DATABASE_URL`	min(10, 40 / процессов), не меньше 3
`METRICS_TOKEN`	`server`	Если задан, `/metrics` требует `Authorization: Bearer <токен>`	—
`EVENTS_POLL_MS`	`server`	Как часто процесс проверяет новые события для потока SSE, мс	2000
`REPLICATION_PASSWORD`	реплика	Пароль роли `replicator`; обязателен для `docker-compose.replica.yml`	—

ПЕРЕМЕННАЯ	ГДЕ	НАЗНАЧЕНИЕ	ПО УМОЛЧАНИЮ
`BASEBACKUP_INTERVAL`, `BASEBACKUP_KEEP`	`basebackup`	Интервал физических копий, секунд; сколько копий хранить	86400, 7
`TRUST_PROXY`	`server`	Каким прокси доверять при определении адреса клиента (влияет на ограничение частоты входа)	`loopback, linklocal, uniquelocal`
`PORT`	`server`	Порт API внутри сети compose	8080
`BACKUP_INTERVAL`	`backup`	Интервал резервного копирования, секунд	86400
`BACKUP_KEEP_DAYS`	`backup`	Срок хранения копий, дней	14
`SEED_DEMO`	seed	`0` — не создавать демо-заявки на согласовании	—
`APP_URL`	`server`	Внешний адрес планировщика: ссылки в письмах, адрес возврата единого входа	`http://localhost`
`SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`	`server`	Почтовый релей; без `SMTP_HOST` письма не отправляются. `SMTP_SECURE=true` — TLS сразу (465), иначе STARTTLS	—, 587, `false`
`SMTP_USER`, `SMTP_PASSWORD`	`server`	Учётная запись релея, если он требует входа	—
`MAIL_FROM`	`server`	Отправитель писем	`Планировщик отпусков <noreply@localhost>`
`OIDC_ISSUER`, `OIDC_CLIENT_ID`, `OIDC_CLIENT_SECRET`	`server`	Единый вход OpenID Connect; без `OIDC_ISSUER` — только вход по паролю	—

ПЕРЕМЕННАЯ	ГДЕ	НАЗНАЧЕНИЕ	ПО УМОЛЧАНИЮ
`OIDC_LOGIN_CLAIM`	`server`	Утверждение с логином сотрудника	`preferred_username`
`OIDC_LABEL`	`server`	Подпись кнопки «Войти через …», винительный падеж	`корпоративную учётную запись`
`LOCAL_LOGIN`	`server`	`false` — вход по паролю только для администраторов (аварийный)	`true`

Обратный прокси. Прокси организации должен передавать `X-Forwarded-For` и находиться в частной сети (10/8, 172.16/12, 192.168/16). Если прокси с публичным адресом, укажите его в `TRUST_PROXY`, иначе ограничение частоты входа будет считать всех пользователей одним клиентом.

HTTPS. Cookie сессии получает признак `Secure` в production — стенд должен открываться только по HTTPS.

Поток событий через прокси. Интерфейс получает уведомления и изменения графика потоком `GET /api/v1/events` (Server-Sent Events). Прокси организации не должен буферизовать этот адрес и обрывать его раньше чем через минуту простоя (сервер шлёт пустую строку раз в 25 секунд). Для nginx:

```
location = /api/v1/events {
  proxy_pass http://<стенд>;
  proxy_http_version 1.1;
  proxy_set_header Connection "";
  proxy_buffering off;
  proxy_read_timeout 1h;
}
```

Если поток не проходит, интерфейс работает как раньше — опрашивает уведомления раз в 30 секунд, — но изменения графика видны только после обновления страницы.

Заголовки безопасности. nginx контейнера `web` отдаёт Content Security Policy и запрет встраивания во фреймы. Не добавляйте на прокси собственную CSP, которая разрешает inline-скрипты.

5 Почта

1. Получите у администраторов почты адрес релея и, если требуется, учётную запись для отправки.
2. Задайте в `.env` `SMTP_HOST`, `SMTP_PORT`, `MAIL_FROM`, `APP_URL` и при необходимости `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_SECURE`; перезапустите `server`.
3. Укажите свой email в карточке администратора и нажмите «Профиль и настройки → Проверочное письмо себе».

Письма ставятся в очередь в той же транзакции, что и событие, и отправляются фоном: недоступность почты не мешает работе, неотправленные письма повторяются через 1, 5, 30 минут, 2 и 12 часов. Состояние очереди:

```
docker compose exec postgres psql -U vacation -d vacation -c "select status, count(*) from email_outbox group by status"
```

6 Единый вход

Планировщик подключается к любому провайдеру OpenID Connect: Keycloak, ADFS (Windows Server 2016+), Blitz Identity Provider и другим.

1. Зарегистрируйте у провайдера конфиденциальное приложение: тип «authorization code», адрес возврата `https://<APP_URL>/api/v1/auth/oidc/callback`, области `openid profile email`.
2. Задайте `OIDC_ISSUER` (адрес издателя: у Keycloak — `https://sso.company.ru/realms/<realm>`, у ADFS — `https://adfs.company.ru/adfs`), `OIDC_CLIENT_ID`, `OIDC_CLIENT_SECRET`.
3. Выберите `OIDC_LOGIN_CLAIM` — утверждение, совпадающее с логином в планировщике: `preferred_username` (Keycloak), `upn` или `email` (ADFS). Часть до «@» и после «» отбрасывается: `ivanov.ii@company.ru` и `COMPANY\ivanov.ii` дают `ivanov.ii`.
4. Перезапустите `server`. На экране входа появится кнопка единого входа.
5. Когда все сотрудники входят через провайдера, задайте `LOCAL_LOGIN=false`: пароль останется только у администраторов — на случай недоступности провайдера.

Сотрудника, которого нет в планировщике или который уволен, единый вход не пускает: заводите сотрудников через «Сотрудники → Импорт».

7 Интеграция с кадровой системой

Импорт сотрудников принимает не только файлы, но и JSON — так кадровая система или скрипт выгрузки синхронизирует сотрудников и остатки по расписанию:

```
curl -s -X POST "https://otpusk.company.ru/api/v1/imports/employees?dryRun=false" \
  -H "Authorization: Bearer <access-токен служебной учётной записи кадровой службы>" \
  -H "Content-Type: application/json" \
  -d '{"rows":[{"login":"ivanov.ii","name":"Иванов Иван Иванович","dept":"ИТ
служба","carryOver":3.5}]}'
```

С `dryRun=true` ответ содержит только план (отчёт сверки) без изменений. Повторная отправка тех же данных ничего не меняет. Токен скрипт получает входом `POST /api/v1/auth/login` служебной учётной записи с правами кадровой службы; при `LOCAL_LOGIN=false` вход по паролю есть только у администраторов — тогда заведите для интеграции отдельную учётную запись администратора и храните её пароль в хранилище секретов.

8 Проверка

```
docker compose --profile full ps                                    # все сервисы
running/healthy
curl -s http://localhost/api/v1/employees -o /dev/null -w '%{http_code}\n'   # 401 — API за
nginx отвечает
docker compose exec server wget -qO- http://localhost:8080/readyz  # {"ok":true} — БД
доступна
docker compose exec server wget -qO- http://localhost:8080/metrics | grep -E
'^vp_(db_up|process_count)'   # vp_db_up 1, число процессов
docker compose logs backup --tail 5                                 # backup:
/backups/vacation-….dump
```

После этого войдите в интерфейс и откройте «График отпусков», «Справочники → Производственный календарь» (календарь года загружен) и «Журнал аудита».

9 Обновление

1. Прочитайте `docs/versions.md` и `CHANGELOG.md` новой версии: особые шаги и изменения схемы. Для 0.4.0: миграции добавляют роль кадровой службы, email сотрудников и очередь писем, а также исправляют демо-календари 2026–2027 годов, если их не меняли вручную. Для 0.5.0: миграции добавляют таблицы `data_version` (с триггерами на таблицах графика) и `login_attempts`; `server` запускает по процессу на ядро (`WEB_CONCURRENCY=auto`) — проверьте, что `max_connections` PostgreSQL не меньше 100; если прокси организации стоит перед стендом, настройте поток событий (раздел «Настройка»).
2. Сделайте внеочередную резервную копию (раздел «Резервное копирование»).
3. Получите новую версию и пересоберите стенд:

```
git fetch --tags && git checkout v0.x.y
docker compose --profile full up -d --build
```

4. Миграции применятся при старте `server`. Выполните проверку.

Откат: вернитесь на предыдущий тег и восстановите копию, снятую перед обновлением, — миграции не откатываются автоматически.

10 Резервное копирование

Сервис `backup` снимает копию `pg_dump` в формате custom раз в `BACKUP_INTERVAL` секунд и хранит её `BACKUP_KEEP_DAYS` дней в томе `db-backups`. Файл сначала пишется во временный и переименовывается только после успешного завершения, поэтому в томе нет оборванных копий.

```
# Список копий
docker compose exec backup ls -lh /backups
# Внеочередная копия
docker compose exec backup sh -c 'pg_dump -Fc -f /backups/vacation-manual-$(date +%Y%m%d-%H%M).dump'
# Копия на хранилище вне сервера
docker compose cp backup:/backups/<файл>.dump ./
```

Храните копии и вне сервера: том `db-backups` находится на том же диске, что и база.

10.1 Восстановление

```
docker compose --profile full stop server web
docker compose exec postgres dropdb -U vacation vacation
docker compose exec postgres createdb -U vacation vacation
docker compose cp ./<файл>.dump postgres:/tmp/restore.dump
docker compose exec postgres pg_restore -U vacation -d vacation /tmp/restore.dump
docker compose --profile full start server web
```

10.2 Ежемесячная проверка восстановления

Раз в месяц восстановите последнюю копию в отдельную базу и сверьте данные:

```
docker compose exec postgres createdb -U vacation restore_check
docker compose exec backup sh -c 'pg_restore -d restore_check $(ls -t /backups/*.dump | head -1)'
docker compose exec postgres psql -U vacation -d restore_check -c 'select count(*) from employees; select count(*) from vacation_requests;'
docker compose exec postgres dropdb -U vacation restore_check
```

Результат проверки фиксируйте в журнале эксплуатации.

11 Обслуживание

Производственный календарь на новый год. Кадровая служба вносит его в «Справочниках → Производственный календарь»: отмечает праздники в сетке года или загружает файл xmlcalendar.ru. SQL не требуется.

Первый администратор на рабочем контуре. Получите scrypt-хеш временного пароля средствами самого сервера:

```
docker compose exec server node -e
"require('./dist/auth/passwords').hashPassword(process.argv[1]).then(console.log)"
'<временный пароль>'
```

и создайте учётную запись:

```
INSERT INTO employees (login, password_hash, name, is_admin, updated_at)
VALUES ('admin', '<хеш из предыдущей команды>', 'Администратор системы', true, now());
```

После первого входа смените пароль в интерфейсе и заведите остальных сотрудников в разделе «Сотрудники».

Журналы. `docker compose logs -f server` — API; `web` — запросы nginx; `backup` — результат копирования.

12 Производительность

Один процесс Node.js использует одно ядро, поэтому `server` запускает `WEB_CONCURRENCY` процессов (`auto` — по числу ядер, доступных контейнеру). Если контейнеру выделена часть ядер квотой (`cpus:` в compose), задайте число явно — `auto` видит все ядра узла. Процессы одного экземпляра вместе держат не больше ~40 соединений с БД; при втором экземпляре на другом узле — до 80, что укладывается в `max_connections = 100`.

Второй экземпляр `server` ставится на другой узел с тем же `.env` и той же БД; балансировщик организации распределяет запросы между узлами без привязки сессий. Лимит входа, очередь писем и кэш графика согласованы через PostgreSQL — дополнительной настройки не требуется.

Нагрузочная проверка. Перед вводом в эксплуатацию на новом оборудовании повторите прогон версии на отдельной БД (не на рабочей):

```
cd server
DATABASE_URL=postgresql://vacation:vacation@localhost:5432/vacation_load npx prisma migrate
deploy
DATABASE_URL=postgresql://vacation:vacation@localhost:5432/vacation_load npm run
db:seed:load   # 5000 сотрудников
# сервер на этой БД, затем:
node ../tools/load/run.mjs --url http://<сервер>:8080 --users 500 --ramp 60 --duration 900
```

Скрипт наполнения откажется работать с БД, в имени которой нет `load`. Генератор печатает таблицу p50/p95/p99 по операциям; цели — p95 чтения ≤ 300 мс, записи ≤ 500 мс, без ошибок.

13 Мониторинг

Сервер отдаёт метрики Prometheus на `GET /metrics` порта 8080. nginx стенда этот адрес наружу не проксирует; если Prometheus обращается к серверу через общую сеть, задайте `METRICS_TOKEN`.

```
docker compose --profile full --profile monitoring up -d     # Prometheus на 127.0.0.1:9090
```

Конфигурация — `deploy/monitoring/prometheus.yml`, правила оповещений — `deploy/monitoring/alerts.yml`. Если в организации уже есть Prometheus, перенесите в него задание `vacation-planner` и файл правил. Панель Grafana `deploy/monitoring/grafana-dashboard.json` импортируйте в Grafana организации («Dashboards → Import», источник — ваш Prometheus).

МЕТРИКА	ЧТО ПОКАЗЫВАЕТ
`vp_http_request_duration_seconds`	Длительность запросов по шаблону маршрута — p50/p95 в панели
`vp_http_requests_total`	Запросы по маршруту и коду ответа: доля 5xx
`vp_eventloop_lag_p99_seconds`	Задержка процессов: больше 0,2 с — не хватает ядер
`vp_db_connections`, `vp_db_max_connections`	Соединения с БД по состояниям и предел
`vp_db_errors_total`	Перегрузка БД: пул исчерпан, транзакция не началась, БД недоступна
`vp_mail_outbox`, `vp_mail_outbox_oldest_seconds`	Очередь писем и возраст самого старого неотправленного
`vp_db_replica_lag_seconds`, `vp_db_replication_slot_active`	Отставание и подключение реплики
`vp_db_wal_archive_failing`	Архивирование WAL завершается ошибкой

ОПОВЕЩЕНИЕ	ВАЖНОСТЬ	ДЕЙСТВИЕ
`VacationApiDown`, `VacationDatabaseDown`	критично	Проверьте контейнеры `server`, `postgres`, место на диске
`VacationServerErrors`	предупреждение	Журнал `server`: «Необработанная ошибка»

ОПОВЕЩЕНИЕ	ВАЖНОСТЬ	ДЕЙСТВИЕ
`VacationReadLatencyHigh`, `VacationWriteLatencyHigh`	предупреждение	Панель «Задержка p95 по маршрутам»; задержка цикла событий и соединения БД
`VacationDatabaseOverloaded`, `VacationConnectionsNearLimit`	предупреждение	Увеличьте `max_connections` или уменьшите `DB_POOL_SIZE`, проверьте долгие запросы в `pg_stat_activity`
`VacationReplicaDisconnected`, `VacationReplicaLagging`	предупреждение	Раздел «Реплика и архив WAL»
`VacationWalArchiveFailing`	критично	Место на томе `wal-archive`, журнал `postgres` («archive command failed»)
`VacationMailStuck`, `VacationMailFailed`, `VacationDigestNotRun`	предупреждение	Раздел «Почта»

14 Реплика и архив WAL

Стенд с репликой теряет при аварии основного сервера секунды изменений, а если потеряны и основной сервер, и реплика, — не больше 5 минут (по архиву WAL и физическим копиям). Переключение на реплику занимает около 15 минут по инструкции ниже.

1. Задайте в `.env` пароль репликации:

```
REPLICATION_PASSWORD=<результат openssl rand -base64 32>
```

2. Запустите стенд с дополнением:

```
docker compose -f docker-compose.yml -f docker-compose.replica.yml --profile full up -d
```

При первом запуске реплика создаёт на основном сервере роль `replicator` и слот `replica1`, копирует данные и далее получает поток WAL. Основной сервер архивирует сегменты WAL в том `wal-archive`, сервис `basebackup` раз в сутки снимает физическую копию и удаляет из архива то, что старше самой старой из 7 хранимых копий. Разместите том `wal-archive` на отдельном диске или хранилище — иначе авария диска уничтожит и базу, и архив.

3. Проверьте:

```
C="docker compose -f docker-compose.yml -f docker-compose.replica.yml"
$C exec postgres psql -U vacation -Atc "select application_name, state from
pg_stat_replication"   # replica1|streaming
$C exec postgres psql -U vacation -Atc "select archived_count, failed_count from
pg_stat_archiver"  # failed_count = 0
$C exec basebackup ls /wal-archive/base
# копии по датам
```

14.1 Переключение на реплику

Выполняйте, когда основной сервер недоступен и не вернётся в ближайшее время. Не включайте старый основной сервер, пока реплика работает основным: два основных сервера разойдутся.

```
C="docker compose -f docker-compose.yml -f docker-compose.replica.yml"
$C stop postgres                                                    # если ещё работает
$C exec postgres-replica psql -U vacation -c "select pg_promote()"      # t — реплика стала
основной
DB_HOST=postgres-replica $C --profile full up -d --no-deps server backup basebackup
```

Запишите `DB_HOST=postgres-replica` в `.env`, чтобы следующий `up` не вернул сервер на старый узел. Проверьте вход и график (раздел «Проверка»). Повышенная реплика сама продолжает архивировать WAL на новой линии времени. Переключение отрабатывается в CI на каждый

push (задание «Реплика PostgreSQL, архив WAL и переключение»).

Возврат к схеме с репликой. Бывший основной сервер не подключается обратно: его данные отстали и разошлись. Удалите его том и поднимите как новую реплику повышенного сервера (поменяйте роли сервисов местами в `docker-compose.replica.yml` или разверните реплику заново на исправном узле) — скрипт `deploy/postgres/replica.sh` сделает копию сам.

14.2 Восстановление на момент времени

Если повреждены данные (ошибочное массовое изменение), а не сервер, восстановите физическую копию и примените архив WAL до нужного момента в отдельный каталог:

```
C="docker compose -f docker-compose.yml -f docker-compose.replica.yml"
$C --profile full stop server
mkdir -p /srv/restore && cd /srv/restore
docker run --rm -v vacation-planner_wal-archive:/wal-archive -v "$PWD":/restore postgres:17-
alpine sh -c '
  base=$(ls /wal-archive/base | grep -E "^[0-9]{8}-[0-9]{6}$" | sort | tail -1)
  tar xzf /wal-archive/base/$base/base.tar.gz -C /restore
  touch /restore/recovery.signal
  echo "restore_command = '"'"'cp /wal-archive/%f %p'"'"'" >> /restore/postgresql.auto.conf
  echo "recovery_target_time = '"'"'2026-11-20 14:55:00+03'"'"'" >>
/restore/postgresql.auto.conf
  chown -R 70:70 /restore && chmod 700 /restore'
```

Запустите PostgreSQL на восстановленном каталоге с подключённым архивом (`vacation-planner` — имя проекта compose, по умолчанию — имя каталога стенда):

```
docker run -d --name vp-restore -v /srv/restore:/var/lib/postgresql/data \
  -v vacation-planner_wal-archive:/wal-archive:ro postgres:17-alpine
docker logs -f vp-restore        # «recovery stopping before commit …», «pausing at the end
of recovery»
docker exec vp-restore psql -U vacation -d vacation -c "select count(*) from
vacation_requests"
docker exec vp-restore psql -U vacation -c "select pg_wal_replay_resume()"   # данные верны
— завершить восстановление
```

Затем либо выгрузите нужные данные (`pg_dump` из `vp-restore`) и загрузите их в рабочую БД, либо сделайте этот каталог основным сервером и заново создайте реплику. Без `recovery_target_time` восстановление идёт до последнего заархивированного сегмента. Проверяйте процедуру раз в квартал.

15 Устранение неполадок

ПРИЗНАК	ПРИЧИНА И ДЕЙСТВИЕ
`server` перезапускается, в журнале «JWT_SECRET»	Секрет не задан или короче 32 символов — задайте в `.env`
`/readyz` не отвечает	PostgreSQL недоступен — `docker compose ps postgres`, свободное место на диске
У всех пользователей «Слишком много попыток входа»	Прокси с публичным адресом не входит в `TRUST_PROXY` — все запросы выглядят одним клиентом
После обновления пустой экран	Кеш браузера держит старый `index.html` за прокси — `index.html` отдаётся с `no-cache`, проверьте, что прокси не кеширует его
Дни отпуска считаются календарными	Не загружен производственный календарь года — «Справочники → Производственный календарь»
Проверочное письмо не отправлено	Текст ошибки — ответ почтового сервера: проверьте `SMTP_HOST`, порт, `SMTP_SECURE` и доступ к релею из сети стенда
Кнопки единого входа нет	Не заданы `OIDC_ISSUER` и `OIDC_CLIENT_ID`
«Сервис единого входа недоступен»	Сервер не получает `${OIDC_ISSUER}/.well-known/openid-configuration` — сеть, DNS, сертификат провайдера
«Не удалось подтвердить вход у провайдера»	Неверный `OIDC_CLIENT_SECRET`, адрес возврата или часы сервера расходятся больше чем на минуту; подробности — `docker compose logs server`
«Учётная запись не найдена» у всех	`OIDC_LOGIN_CLAIM` указывает не на логин — проверьте утверждения токена у провайдера
`backup: ОШИБКА` в журнале	Нет места на томе или нет доступа к БД — проверьте `docker compose logs postgres`
«Сервис перегружен. Повторите через несколько секунд.» (`503`)	Не хватает соединений с БД или БД недоступна: метрики `vp_db_errors_total` и `vp_db_connections`; долгие запросы — `pg_stat_activity`

ПРИЗНАК	ПРИЧИНА И ДЕЙСТВИЕ
Уведомления приходят с задержкой до 30 секунд, график не обновляется сам	Прокси организации буферизует или обрывает поток `/api/v1/events` — раздел «Настройка»
`server` не стартует: `connection_limit`, «too many clients»	`WEB_CONCURRENCY` × пул больше `max_connections` — задайте `DB_POOL_SIZE` или уменьшите число процессов
Реплика не стартует, «password authentication failed for user “replicator”»	`REPLICATION_PASSWORD` в `.env` изменился после первого запуска — верните прежний или удалите том `db-replica`, чтобы реплика создалась заново
`basebackup: ОШИБКА`	Реплика ещё не создала роль `replicator` или нет места на томе `wal-archive`